

7

Untangling Complex Flows in React Native Apps with XState

In the previous chapter, we took a look at **MobX**—the second most popular state management library in the **React** ecosystem. MobX introduced some new concepts, such as using state values derived by the state manager. Other high-level concepts were similar to **Redux**—such as presenting state as plain **JavaScript** objects. We will now look at the first outlier on the React-state-management horizon: **XState**. XState treats the state not as an object, but as a finite machine. Don't worry if you haven't heard that term yet, as we'll go over the topic of finite machines in the first section of this chapter.

We will start by looking at the theoretical side of XState's basic idea: state machines. We will then talk about other high-level concepts of XState—state charts, actions, and the XState visual-

izer. When we're comfortable with the theory, we'll configure XState in the Funbook app, and then we will implement XState for managing liking images in the app.

Here's a complete list of what is covered in this chapter:

- What are finite state machines?
- What is XState—high-level concepts
- Configuring XState in the Funbook app
- Using XState for the **FavoritedImages** surface

By the end of this chapter, you will be able to understand and use XState as the state management solution for your projects. You will understand what a state machine is and how it differs from state objects used in other state management libraries. I hope that you will also start seeing which solutions you prefer using.

Technical requirements

In order to follow along with this chapter, you will need some knowledge of JavaScript and **ReactJS**. If you have followed at least *Chapters 1* through *4* of this book, you should be able to go forward without any issues.

Feel free to use an IDE of your choice, as React Native does not need any specific functionality. Currently, the most popular IDEs for frontend developers are Microsoft's VS Code, Atom, Sublime Text, and WebStorm.

The code snippets provided in this chapter are here to illustrate what we should be doing with the code—they do not provide the whole picture. For a better experience of coding along, please open the GitHub repo in your IDE and look at the files in there. You can start with the files in either the folder named **example-app-full** or **chapter-7**. If you start with **example-app-full**, you will be responsible for implementing the solutions described in this chapter. If you choose to look at **chapter-7** you will see the entire solution implemented by me.

If you get stuck or lost, you can check the code in the GitHub repo:

<https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-7>.

What are finite state machines?

If I were to ask you, my dear reader, to guess what finite state machines are, you would probably say they are related to managing state in applications. After all, this whole book is on that topic!

Funny thing is, finite state machines have nothing to do with applications; they have nothing to do with React or even programming at all. A finite state machine is a mathematical model of computation. It's an abstract concept that can be applied to real-life objects or problems, and it represents a machine that can be in exactly one of a finite number of states at any given time. The predefined states can be changed from one to another in response to some user input. Traffic lights are an example of a simple finite state machine: a traffic light can be green, red, or yellow at any time, and it should never display two colors at once. Another example of a simple state machine is an elevator. The default state of an elevator is to stand still with doors closed. When a user pushes the button summoning the elevator, the elevator transitions to a state of movement. When it reaches the right floor, it opens and closes the doors. The elevator then goes back to the default, idle state, waiting for the next user input.

If you wish to find out more about this theoretical concept, you will find a link to a very thorough *Wikipedia* page on finite state machines in the *Further reading* section. As for this book,

it's time to find out why we are talking about this concept at all. Can you guess? I bet you can! Finite state machines are the basic concept of the state management library we're analyzing in this chapter: XState.

What is XState – high-level concepts

Now that we have a grasp on the theoretical concept of finite state machines, we can move on to talking about XState and its main concept: finite state machines! But this time, we'll look at it in the world of programming global state in applications.

When using XState to manage global state in an application, we should think of our state as a finite state machine. This means abandoning the previous concept of representing state as a plain JavaScript object. With XState, a component—or a surface—is a machine that can be in one of multiple predefined states. Let's consider the user login flow. Our entire app can be in one of two states: the user is logged in or the user is not logged in. We would also need a transition mechanism for the user to move from one of the states to the other. The same goes for images on the **Home** surface. Every image is either in the state of being “liked” or “not liked”. The user can change the

current state of the image by clicking the heart icon below the image.

Besides finite state machines, there are two other important concepts used in XState: **statecharts** and the **actor model**. Statecharts are basically drawings that can be used to represent state machines. Here's an example of a statechart representing the state and transitions of a light bulb:

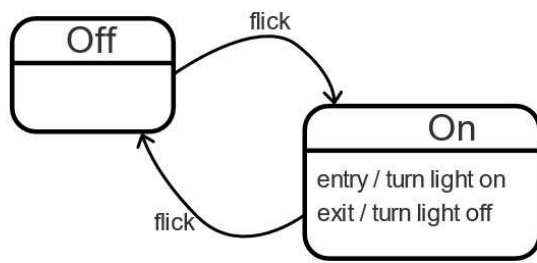


Figure 7.1 – Simple statechart drawing of a light switch

The preceding diagram serves as a very simple state machine. When working on a mobile application, you may find yourself working on much more complicated state machines. Starting from something as trivial as a form, you can find yourself adding multiple states, such as enabled/disabled, valid/invalid, and clean/dirty, on multiple elements. Without statecharts, you would face a state explosion. As fun as it may sound, it's not great to face in an app. Let's take a look at the example of complex inputs drawn out with state transitions:

The last big idea behind XState is actor models. This is a mathematical model of computation, stating that everything is an “actor” and can do three things: receive messages, send messages, and do something with the received messages.

I was very lucky to be able to ask XState’s author, *David Khourshid*, a few questions on the topic of his state management library. He told me he “*created XState for two reasons: to manage complex logic and to visualize complex logic. State machines and statecharts are visual formalisms that excel in representing even the most complex flows and logic in a visually clear way, and I wanted a simple way to use them in JavaScript applications.*” He added that the high-level ideas of XState were heavily influenced by the **World Wide Web Consortium (W3C) State Chart XML (SCXML)** specification.

Let’s take a quick detour to find out what SCXML is and what it means that it has a W3C specification. Depending on your experience in programming, you may have heard of the **Extensible Markup Language (XML)** file format and markup language. XML is used to store, transmit, and reconstruct data. XML files are easy to read when properly indented and formatted, as they simply describe data. SCXML is a cousin of XML. It’s an XML-based markup language used for providing a state-machine-based environment. The fact that it has a W3C specification means that it can be used for various internet-related

programs with great confidence. You can find a link to the entire W3C specification in the *Further reading* section.

Going back to XState, not only has it been influenced by SCXML, but it is also fully compatible with it, which means you could write an SCXML document describing states, and it will work with an XState implementation in your React Native app. You can also write it in JavaScript. Whatever rocks your boat!

I asked David Khourshid about the future of his library. XState is an open source project, as with all the other state management libraries we talk about in this book. David said maintaining XState and working on XState-related tools is his full-time job. He is working on new and powerful collaborative editing tools for the XState visualizer. He said: *“the next major version of XState (version 5) will have many more features, be more modular, and have “actors” as first-class citizens. Actors are entities that can send and receive messages, and state machines are just one of many behaviors that an actor can have. You can also represent actors as promises, observables, reducers, and more, which will allow developers to use XState’s API (and visual tools) for all of their logic, not just the state-machine-specific logic.”*

You may have noticed a mention of an XState visualizer in the previous paragraph. This tool is something that absolutely sets

XState apart from other state management libraries. Thanks to this visualizer, you can see a graphical representation of states and transitions between states in your app. You can use it to plan a new app or debug an app that you are working on. You can find the visualizer at <https://xstate.js.org/viz/>. Here's an example screenshot of what it looks like:

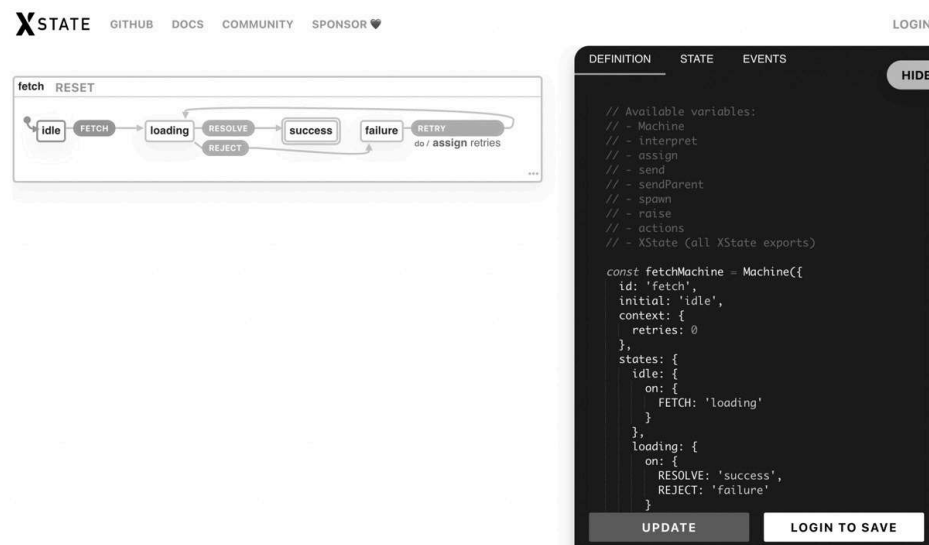


Figure 7.3 – Screenshot of the XState visualizer

David said that the visualizer is one of the hardest things he has worked on. It's always a work in progress, and it has gone through many iterations. Currently, it's an "SVG-based 'canvas' with HTML inside." Even though it's somewhat interactive right now—you can click on transitions and observe how the state changes—David said that *"making it interactive is yet another layer of difficulty, especially for drag-and-drop interactions and*

modifying the statechart." Personally, I'm very excited about the new versions of the visualizer. It has helped me many times to plan the best possible state machine for the apps I worked on (that used XState).

In this section, we have talked about the main ideas behind XState. They are different from all the previous approaches we analyzed. The entire library is based on the mathematical concept of finite state machines. It also uses the theories behind statecharts and actor models, to make sure managing state in a complex app can be done effectively. Now, it's time to see this library in action. Let's move on to implementing XState in the Funbook app.

Configuring XState in the Funbook app

Let's see what it takes to use XState in a real app. If you would like to follow along on your own, you can copy the **example-app-full** folder and use it as a starting point. If you prefer to look at the code related to this chapter, please look in the **chapter-7** folder: <https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-7>.

First things first—we need to add XState to the project. You can do so by running one of the two following commands:

```
npm install xstate@latest --save
// or
yarn add xstate@latest --save
```

XState itself is an unopinionated library, much like MobX. This means it is not ready out-of-the-box to work with React. The XState documentation has a section called *Recipes* where you can read more on the implementation with React or other UI libraries, such as Vue or Svelte. As for us, we need to add the React-related dependency, **xstate-react**. Let's do this by running one of the two following commands:

```
npm install xstate-react@latest --save
// or
yarn add xstate-react@latest --save
```

Now that we have the dependencies ready, let's run the app to make sure everything is working as expected. If everything is OK, we can create our very first state machine. We will start with a simple example: user login flow. At a high level, there isn't much logic involved in this flow. The user can be either logged in or out, and they transition from one state to the other and back:


```
import { createMachine } from 'xstate';
export const userFlowMachine = createMachine({
  id: 'userFlow',
  initial: 'anonymous',
  states: {
    anonymous,
    authenticated,
  }
});
```

Reading through the code is rather logical. We start by importing a `createMachine` function, which we call to create our `userFlowMachine` instance. In `userFlowMachine`, we start by defining the machine ID and the initial state. We then continue to define the two possible states of the app. The user in our app can be anonymous or authenticated. But how can the user transition from one state to the other? Let's add this functionality to the state machine:

```
import { createMachine } from 'xstate';
export const userFlowMachine = createMachine({
  id: 'userFlow',
  initial: 'anonymous',
  states: {
    anonymous: {
      on: {
        LOGIN: { target: 'authenticated' },
      }
    }
  }
});
```

```
    },  
    authenticated: {  
      on: {  
        LOGOUT: { target: 'anonymous' },  
      }  
    },  
  },  
});
```

Great! Now, the user can be in the **anonymous** state, from which they can transition using the **LOGIN** transition. At this moment, they will be in the **authenticated** state, from which they can transition using the **LOGOUT** transition. You could continue improving this example by adding some implementation details to the **LOGIN** and **LOGOUT** transitions, or maybe an error state. But I will stop talking about this particular state machine now and see how it should be used in a React app.

Unsurprisingly, the XState docs recommend using React Context to manage global state with XState. Luckily for us, we have a good handle on React Context by now, right? So, let's look at an example of React Context in the XState documentation:

```
import React, { createContext } from 'react';  
import { useInterpret } from '@xstate/react';  
import { userFlowMachine } from './machines/userFlowMachine';  
export const GlobalStateContext = createContext({});
```

```
export const GlobalStateProvider = (props) => {  
  const userFlowService = useInterpret(userFlowMachine);  
  return (  
    <GlobalStateContext.Provider value={{ userFlowService }}>  
      {props.children}  
    </GlobalStateContext.Provider>  
  );  
};
```

Hmm... what is this **useInterpret()** function? It's imported from **xstate-react**, and it's a special tool to make sure we don't cause too many re-renders when using React Context. **useInterpret()** returns a service, which is a reference to the state machine. As per the XState documentation: *"this value never changes, so we don't need to worry about wasted re-renders."*

KNOWING YOUR TOOLS

Every tool was created with an idea of how it should be used. You could take a hammer and use the wooden handle to hit a nail, but you have learned this is not how hammers work best. The same rule applies to JavaScript libraries. No one was born with the knowledge of JavaScript libraries and tooling. We all must read the documentation and learn our tools' best practices.

We have a way to create context, so now, let's go through XState's instructions for using it. We will have to subscribe to

the service of the global context we defined at the root of the app. Here's what such a subscription would look like:

```
import React, { useContext } from 'react';
import { GlobalStateContext } from '../globalState';
import { useActor } from '@xstate/react';
export const SomeComponent = (props) => {
  const globalServices = useContext(GlobalStateContext);
  const [state] = useActor(globalServices.userFlowService);
  return state.matches('loggedIn') ? 'Logged In' : 'Logged Out';
};
```

We have completed the basic setup for XState in a React Native app. There are many paths to be taken now: improving performance, dispatching events, or using state selectors. We will go over the necessary steps in the next section, where we will set up XState for the **LikedImages** surface and the modal responsible for adding liked images.

Using XState for the FavoritedImages surface

In the previous section, we set up a basic machine that could be used to control the user flow in the app. Let's add a new machine, for our real-world use case: liking images in a social media clone app.

We'll start by creating a machine **minimum viable product (MVP)**:

```
// src/machines/likeImagesMachine.js
import { createMachine } from "xstate";
export const likeImagesMachine = createMachine({
  id: "likeImagesMachine",
  context: {
    likedImages: [
      { Example Image Object 1 },
      { Example Image Object 2 }
      ...
    ],
  },
  initial: "loading",
  states: {
    loading: {},
    ready: {},
    error: {},
  },
});
```

Let's analyze this code from the top: we start by importing the **createMachine** function, which we use on the very first line of the **likeImagesMachine** function. We set the ID of the machine and the context. Bear in mind that XState context is different from React context. We've talked a lot about ReactJS context; we know it can be used to share state between components. XState context is a container for quantitative data (such as

strings, arrays, or objects), which can potentially be infinite. The array of liked images is a great example of this sort of data, and that's why we'll be keeping this array in our machine's context. For testing purposes, we will add a couple of images to this default array of **likedImages** in the context. All that's left is defining states of our machine and setting the default state. Easy-peasy!

We will start by creating and configuring a wrapper for the state, with the help of React's context. Once everything is set up correctly with mock data, we will fetch real data from our backend. Having fetched data, we will write the last piece of code: managing liked images with XState.

Configuring context and components

Now is the time to talk about the first type of context: React context. We set up a nifty context with the user flow in the previous section. We will add the liked images machine to this context:

```
// src/context.js
[...]  
import { useInterpret } from "@xstate/react";  
import { likeImagesMachine } from "../machines/likeImagesMachine";  
import { userFlow } from "../machines/userFlowMachine";  
export const GlobalStateContext = createContext({});
```

```
export const useXStateContext = () => {
  const context = React.useContext(GlobalStateContext);
  if (context === undefined) {
    throw new Error(
      " useXStateContext must be used within a
    );
  }
  return context;
};

export const GlobalStateProvider = (props) => {
  const likedImagesAppService = useInterpret(likeImagesMachine);
  const userFlowService = useInterpret(userFlow);
  const mergedServices = {
    likedImagesAppService,
    userFlowService,
  };
  return (
    <GlobalStateContext.Provider value={mergedServices}>
      {props.children}
    </GlobalStateContext.Provider>
  );
};
```

GlobalStateContextProvider"

This is a good moment to improve the basic context we set up in the previous, more theoretical part of this chapter. We will do so by adding a new custom hook called **useXStateContext**. Using custom hooks with React context is a best practice that we covered in previous chapters. In the **GlobalStateProvider** function, we added **likedImagesMachine** through the **useInterpret** custom hook provided by the good people at XState.

We merge the interpreted machines and pass them as the context value. The last piece of the context value is wrapping the component in context. We will have to keep the global state at the very root of the app so that both the **FavoritedImages** surface and **ImageDetailsModal** can reach it. Here's what your **App.js** should roughly look like:

```
// src/App.js
[...]  
import {  
  [...]  
  GlobalStateProvider  
  } from "../src/context";  
[...]  
return (  
  <SafeAreaProvider>  
    <GlobalStateProvider>  
      <UserStateContext.Provider value={userLoggedIn}>  
[...]
```

Let's use this brand-new machine, interpreted by React context and holding a few example images in its own context, in the **FavoritedImages** surface. The list of favorited images is rendered in the **ListOfFavorites** component, and this is the component we will be changing:

```
// src/components/ListOfFavorties.js  
import { useXStateContext } from "../context";
```



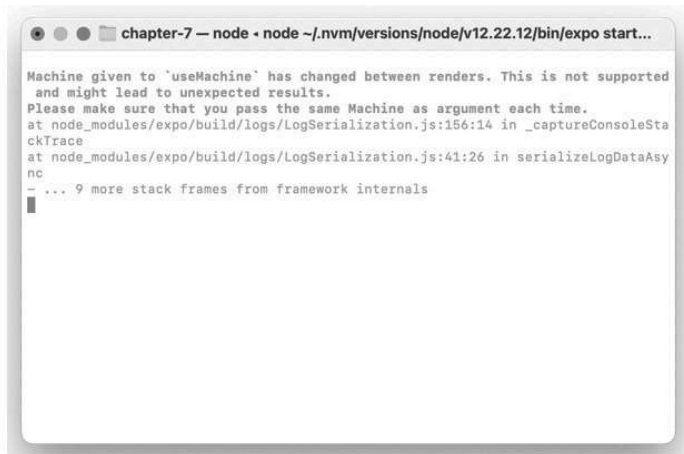
```
import { useActor } from "@xstate/react";
export const ListOfFavorites = ({ navigation }) => {
  const globalServices = useXStateContext();
  const [state] = useActor(globalServices.likedImagesAppService);
  const [imageData, updateImageData] = useState(state.context.likedImages);
  //...
  return (
    //...
    <FlatList
      data={imageData}
    //...
  );
};
```

We start by importing the custom **useXStateContext** hook we created to easily consume React context. The second thing we need to import is XState's **useActor** hook. This is a React hook that subscribes to emitted changes from a given interpreted state machine, named by the XState author "an actor." If you go to the XState documentation, you will find there are other implementations of the **useActor** function, tailored for use with Svelte, Vue, and other libraries. This is because XState, much like MobX, is unopinionated when it comes to UI libraries.

Finally, we need to use all those imported items in our component. We pull in data from React context, and we subscribe to changes through the **useActor** hook. We could use the state returned from the **useActor** hook directly. However, React Native's **FlatList** needs to be informed of data changes very clearly to update. Therefore, I added a **useState** hook, includ-

ing the **updateImageData** setter function, which will come in handy once we try to add images to this array dynamically.

Speaking of dynamically, it's time to think about data fetching through XState. But before we go any further, make sure to run your app with the current changes and make sure you can see the example images from the **likeImagesMachine** function on the **FavoritedImages** surface. If you do encounter any errors, you can look at your terminal window, as many XState errors will be described there. They should also be visible on your phone simulator or physical device. Here's an example error you may see in the console and in the simulator at the same time:

A screenshot of a terminal window titled "chapter-7 — node • node ~/.nvm/versions/node/v12.22.12/bin/expo start...". The terminal displays an error message: "Machine given to 'useMachine' has changed between renders. This is not supported and might lead to unexpected results. Please make sure that you pass the same Machine as argument each time." Below this message, the stack trace is shown, starting with "at node_modules/expo/build/logs/LogSerialization.js:156:14 in _captureConsoleStackTrace" and "at node_modules/expo/build/logs/LogSerialization.js:41:26 in serializeLogDataAsync". The error is followed by "... 9 more stack frames from framework internals".

```
chapter-7 — node • node ~/.nvm/versions/node/v12.22.12/bin/expo start...

Machine given to 'useMachine' has changed between renders. This is not supported
and might lead to unexpected results.
Please make sure that you pass the same Machine as argument each time.
at node_modules/expo/build/logs/LogSerialization.js:156:14 in _captureConsoleSta
ckTrace
at node_modules/expo/build/logs/LogSerialization.js:41:26 in serializeLogDataAsy
nc
... 9 more stack frames from framework internals
```

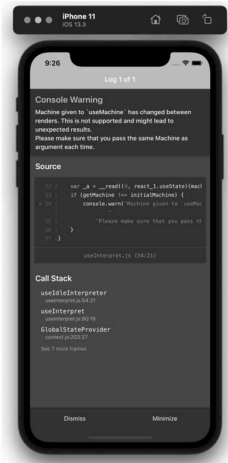


Figure 7.4 – XState errors in the console and phone simulator

Fetching image data

Fetching data isn't always the strong suit of state management libraries. After all, it's not their basic responsibility. In the case of XState, however, fetching comes very naturally as every Promise can be modeled as a state machine. At a high level, we need to kick off a function that will be in a default "loading" state. We'll wait for something to happen with it—either resolve or reject—and go to the appropriate "resolved" or "rejected" states. Here's how our image-fetching machine is shaping up:

```
// src/machines/fetchMachine.js
import { createMachine, assign } from "xstate";
export const fetchImagesMachine = createMachine({
  id: "fetchImages",
```

```
initial: "loading",
context: {
  retries: 0,
  images: [],
},
states: {
  loading: {
    on: {
      RESOLVE: "success",
      REJECT: "failure",
    },
  },
  success: {
    type: "final",
  },
  failure: {
    on: {
      RETRY: {
        target: "loading",
        actions: assign({
          retries: (context, event) => context.retries+1,
        }),
      },
    },
  },
},
});
```

What you can see here is a very simple machine, prepared to describe the process of fetching data from an external source.

We have three states: the initial state of **loading**, and **success** and **failure** states. You can see two actions in the **loading** state, which could be used to manage the fetching mechanism. There's also a **retry** action in the **failure** state. We could use it in the app to let the users manually try to fetch data when an error occurs. As far as the basic setup is concerned, this is all good, but we need to see how to call a real endpoint. In order to do so, we will change the **loading** state:

```
//...
states: {
  loading: {
    invoke: {
      id: 'fetchImagesFunction',
      src: async () => {
        const response = await fetch(
          requestBase + "/john_doe/likedImages.json"
        );
        const imageData = await response.json();
        return imageData;
      },
      onDone: {
        target: "success",
        actions: assign((context, event) => {
          return {
            images: event.data,
          };
        }),
      },
    },
  },
},
```

```
    },  
    onError: {  
      target: "failure",  
      actions: assign({  
        error: (context, event) => "Oops!"  
      }),  
    },  
  },  
},  
},
```

Instead of two actions that could be called manually, I added the **invoke** property to the **loading** state. This way, the images will be loaded automatically when the machine is spawned, without needing user interaction. The **invoke** property's value is an object that contains the **id** and **src** properties of the function that is to be invoked. It is possible to invoke Promises, callbacks—which can send and receive events from the parent machine—observables—which can send events to the parent machine—and entire machines. We will stay on the simple side of things, and we will add an async **fetch** function to the source. You may also create a named function anywhere outside of the machine and invoke it through **src**. We are also using two optional values from the **invoke** property: **onDone** and **onError**. These two transitions come in handy when handling Promises. They act like any other XState transition—they include actions and the target state. Both actions include the **assign** keyword. **assign** is a function that updates the machine's context. We use it here to pass the resulting fetched data to the

Something went wrong",

context so that we can use it later in our app. Assigner functions have some caveats: they have to be pure, and they surrender to a strict order. If you want to read more about them, please check the links provided in the *Further reading* section.

If everything works fine, you should be able to fetch images thanks to this function. But how do we use those images in the **likeImagesMachine** function? Remember that **invoke** property we just used for the Promise? We'll use that same property on **likeImagesMachine** in the loading state, to invoke this fetching machine and pass the fetched data thanks to the **onDone** function:

```
// src/machines/likeImagesMachine.js
import { fetchImagesMachine } from "../fetchImagesMachine";
export const likeImagesMachine = createMachine({
  id: "likeImagesMachine ",
  context: {
    likedImages: [],
    currentImage: null,
  },
  initial: "loading",
  states: {
    loading: {
      invoke: {
        id: "fetchImagesMachine",
        src: fetchImagesMachine,
        onDone: {
```

```
    target: "ready",
    actions: assign({
      likedImages: (context, event) => {
        return event.data.images;
      },
    }),
  },
},
//...
```

In this code snippet, we've imported the **fetchImagesMachine** function and we invoke it in the loading state of the **likeImagesMachine** function. Let's take a closer look at the assigner function we're using to pass image data from **fetchImagesMachine** to this parent machine. It has an **onDone** function, which will be called when **fetchImagesMachine** reaches its final state. This function assigns data returned from the invoked machine to the **context** of **likeImagesMachine**, and the data is passed through **event**. You will notice we're calling **event.data.images**. Where did that come from? This is something we need to add to **fetchImagesMachine**. So far, that machine only passed fetched data to its **context**, but we need to expose it so that the parent machine, **likeImagesMachine**, can access it. We already know the **onDone** event in the parent (**likeImagesMachine**) is called when the child (**fetchImagesMa-**

chine) reaches its final state. The final state in our case is **success**. This is where we can add the **data** property:

```
// src/machines/fetchImagesMachine.js
//...
success: {
  type: "final",
  data: {
    images: (context, event) => context.images,
  },
},
//...
```

This block of code tells the **fetchImagesMachine** function to add a **data** object to its final state. This is the object that we access when we run **onDone** in the parent (**likeImagesMachine**). If everything went well, you should be able to see the entire array of fetched images in your app right now. This is a good moment to run the app on your device or emulator if you haven't already.

Managing the image in the image modal

We have ourselves a nice setup—we're fetching images and feeding them to the app. Our app is rather static, though. We need a way to add new images to the liked images array. We

would also like to check whether an image is liked so that we can display the proper icon in **ImageDetailsModal**.

If we want to know whether an image should be liked or unliked, we first need to know whether it's liked. But even before we can know whether an image has been liked, we need to know all the data pertinent to that image. We will add a new item to the context of the **likeImagesMachine** machine—**currentImage**:

```
export const likeImagesMachine = createMachine({
  id: "likeImagesMachine ",
  context: {
    likedImages: [],
    currentImage: null,
  },
  //...
```

This is where we will store information on the currently viewed image. The context is initialized as **null**, and we need to add an action that will update this context value. We will add a new event called **MODAL_OPEN** to the **ready** state of **likeImagesMachine**:

```
// src/machines/likeImagesMachine
ready: {
  on: {
```

```
MODAL_OPEN: {
  actions: assign((context, event) => {
    return {
      currentImage: event.payload,
    };
  }),
},
MODAL_CLOSE: {
  actions: assign((context, event) => {
    return {
      currentImage: null,
    };
  }),
},
},
//...
```

We will call the **MODAL_OPEN** action when **ImageDetailsModal** is opened, and **MODAL_CLOSE** when the modal is closed—pretty straightforward! You can see the code in action here:

```
// src/surfaces/ImageDetailsModal.js
export const ImageDetailsModal = ({ navigation, route }) => {
  const globalServices = useXStateContext();
  const { send } = globalServices.likedImagesAppService;
  useEffect(() => {
    send({
      type: "MODAL_OPEN",
      payload: route.params.imageItem,
    });
  });
};
```

```
return () => {  
  send("MODAL_CLOSE", {});  
};  
, []);
```

We start by using a custom hook called **useXStateContext** in order to consume context values that we set up previously. Then, we use the **send** function from **likedImagesAppService**. Finally, I've added a **useEffect** hook calling the **MODAL_OPEN** action when the modal is rendered and **MODAL_CLOSE** as a cleanup function.

Now that we have the current image saved in the machine context, we can check whether it's liked or not. To do that, we will use yet another utility from XState: a custom hook called **useSelector**. **Selector** is a name that may sound familiar to you. In JavaScript, there are query selectors, Redux promotes using selector functions, and there are also CSS selectors. XState selectors are the closest ideologically to the ones in Redux. They are special functions that receive the current state and can return a value based on some conditions. Our current state is the array of images and the current image, and the condition is if the current image is in the image array. The code is illustrated in the following snippet:

```
const isImageLikedSelector = (state) => {  
  if (!state.context.currentImage) {
```

```
    return;  
  }  
  const checkIfInImagesArray = state.context.likedImages.find(  
    (image) => image.itemId === state.context.currentImage.  
  );  
  return !!checkIfInImagesArray;  
};
```

As mentioned previously, this selector will receive the current state as the first argument. We start by checking that the images array is not **null**. We are running a **find** function on that array, and if it were **null** or **undefined**, this would cause the app to crash. Once we're sure the images array exists, we can filter it by the current image. You can place this function anywhere you'd like (in the same file as the machine, in a file called **selectors** or **utilities**, and so on) and then import it into **ImageDetailsModal**:

```
// src/surfaces/ImageDetailsModal.js  
export const ImageDetailsModal = ({ navigation, route }) => {  
  const globalServices = useXStateContext();  
  const { send } = globalServices.globalAppService;  
  const isImageLiked = useSelector(  
    globalServices.globalAppService,  
    isImageLikedSelector  
  );  
};
```

The **isImageLiked** constant can be used in the component to check which icon should be displayed and which action (liking

itemId

or unliking) should be called.

Liking images

Our state machines are aware of the array of images we have fetched and that we display on the **FavoritedImages** surface. They also become aware of the currently seen image through the **MODAL_OPEN** action. Now, we need to tell them what to do if someone presses the “like” button. Let’s add a new action to the **likeImagesMachine** function:

```
// src/machines/likeImagesMachine.js
//...
ready: {
  on: {
    LIKE: {
      actions: assign((context, event) => {
        const updateImageArray = event.payload.concat(context.likedImages);
        return {
          likedImages: updateImageArray,
        };
      }),
    },
  },
//...
```

We’re using the assigner function, which we’ve met before. Inside it, we’re concatenating the array containing only the current image to the full array of all images. This way, the

newly added image will be at the top of the array and at the top of **FlatList**. Now that the action is ready, we can call it in the modal, like so:

```
// src/surfaces/ImageDetailsModal
//...
<Pressable
  onPress={() => {
    if (!isImageLiked) {
      send({ type: "LIKE", payload:
    }
  }}
//...
```

We've done quite a few changes—let's test them in our app. If you followed along, you should be able to see that the fetched images load correctly on the **FavoritedImages** surface. The **ImageDetails** modal opens correctly as well, showing a full heart for images that are liked and an empty heart for images that are not liked (those on the **Feed** surface). We can even press the empty heart and it changes to be filled! The like action and the selector work as expected! Great!

Unfortunately, **FlatList** is a little more stubborn. As mentioned before, **FlatList** needs explicit data changes in order to re-render, and we need it to re-render if we want to see the new images added. We will have to “twist its hand” a little bit, by adding this **useEffect** hook:

```
[route.params.imageItem] }));
```

```
// src/components/ListOfFavorites
export const ListOfFavorites = ({ navigation }) => {
  const globalServices = useXStateContext();
  const [state] = useActor(globalServices.globalAppService);
  const [imageData, updateImageData] = useState([]);
  useEffect(() => {
    updateImageData(state.context.likedImages);
  }, [state.context.likedImages]);
  //...
```

And now, everything should work flawlessly! Time for a pat on the back! We've covered a lot of topics in this section. We have gone over a real-world implementation of setting up multiple state machines, invoking fetching functions, passing context values between machines, calling actions, and using selectors. With this knowledge, you should be able to configure any app to use XState as the state management library.

Summary

XState is the first state management library in this book to be fundamentally based on mathematical principles. We talked briefly about those principles, as understanding them is very useful for understanding XState. The most important concept is state machines. They are not new in the world of mathematics; they are quite novel, however, when it comes to how we think about global state in mobile apps. Once we had a grasp

on the theory and we discovered the very useful XState visualizer, we were ready to do real work. We set up XState in the Funbook app, using best practices described in the XState documentation. We covered the topic of implementing XState as the global state solution for the use case of managing liked images. We looked at fetching data and changing data with XState. I hope you enjoyed it! Now, it's time to continue our journey to the next outlier in the state management libraries world: **Jotai**.

Further reading

- <https://brilliant.org/wiki/finite-state-machines/>: Finite state machines.
- <https://www.w3.org/TR/scxml/>: W3C SCXML specification.
- <https://xstate.js.org/docs/recipes/react.html#local-state>: XState recipes.
- <https://xstate.js.org/docs/guides/context.html#assign-action>: Assigner action.